

บทที่ 4

การวิเคราะห์และการออกแบบระบบ

นักพัฒนาโปรแกรมระดับอาชีพจะเริ่มค้นพัฒนาระบบอย่างเป็นขั้นตอนและมีหลักการ คือ ขั้นตอนการวิเคราะห์และออกแบบ ทั้งนี้การศึกษาและการใช้กระบวนการพัฒนาเชิงวัตถุต่างๆ จำเป็นต้องมีความเข้าใจแนวคิดเชิงวัตถุอย่างแม่นยำ ในระบบซอฟต์แวร์ก็เช่นเดียวกัน ผู้ที่จะพัฒนาระบบซอฟต์แวร์ที่ต้องการใช้ภาษาในการเขียนโปรแกรมเชิงวัตถุ (OOP) ก็ควรจะต้องการวิเคราะห์และออกแบบระบบซอฟต์แวร์ด้วยกระบวนการพัฒนาเชิงวัตถุเช่นกัน ปัจจุบันมีวิธีหรือกระบวนการพัฒนาซอฟต์แวร์เชิงวัตถุมากมายหลายวิธี โดยเฉพาะกระบวนการที่มีความเกี่ยวข้องกับภาษาจาวาและแอล

4.1 ขั้นตอนของกระบวนการพัฒนาซอฟต์แวร์เชิงวัตถุ

วัตถุประสงค์ของทุกกระบวนการพัฒนาซอฟต์แวร์ คือ การแปลงความต้องการของผู้ใช้ให้เป็นระบบที่มีคุณภาพและสามารถใช้งานได้จริง อีกทั้งยังช่วยลดระยะเวลาที่ต้องทำการเขียนโปรแกรมและแก้ไขข้อผิดพลาดให้น้อยลงด้วย กระบวนการพัฒนาซอฟต์แวร์เชิงวัตถุประกอบไปด้วย 5 ขั้นตอน

4.1.1 การวิเคราะห์ความต้องการของระบบหรือผู้ใช้ (Requirement Analysis)

การวิเคราะห์ความต้องการของผู้ใช้เป็นการค้นหาขอบข่ายของระบบ และเป็นการเตรียมข้อมูลด้านความสามารถของระบบ (System Function) จากมุมมองภายนอกที่จะต้องถูกพัฒนาโดยไม่คำนึงถึงรายละเอียดหรือกรรมวิธีทางเทคนิคต่างๆ

ในการเริ่มต้นเฟสนี้เป็นการกำหนดข้อตกลงระหว่างผู้ใช้งานกับผู้พัฒนา ซึ่งฝ่ายพัฒนาจะต้องเก็บความต้องการของผู้ใช้อย่างละเอียด เรียกขั้นตอนนี้ว่า User Requirement Elicitation

4.1.2 การวิเคราะห์ระบบ (Domain Analysis)

การวิเคราะห์ระบบ (Domain Analysis) หรือ OOA (OO Analysis) เป็นการวิเคราะห์โครงสร้าง (Structure) และพฤติกรรม (Behavior) ของระบบที่จะทำการพัฒนาซึ่งจะถูกนำไปกำหนดรายละเอียดเชิงเทคนิคในเฟสการออกแบบและถูกสร้างจริงเป็นลำดับต่อไปในการอิมพลีเมนต์ระบบ

4.1.3 การออกแบบระบบ (Design)

การออกแบบระบบ (Design) หรือ OOD (OO Design) เป็นการคิดค้นวิธีแก้ปัญหาหรือพิจารณารายละเอียดเชิงเทคนิคเพื่อเตรียมที่จะอิมพลีเมนต์ระบบขึ้นจริง ซึ่งเป็นการนำผลการวิเคราะห์จากเฟสสองมาทำการแก้ไขเพิ่มเติมรายละเอียดเชิงเทคนิคเพื่อสามารถที่จะถูกนำไปสร้างขึ้นเป็นระบบซอฟต์แวร์จริงได้อย่างสมบูรณ์ ดังนั้นในเฟสนี้จะต้องอาศัยความรู้ทางเทคนิคและเทคโนโลยีต่างๆ ที่มีอยู่เพื่อสามารถเลือกได้อย่างเหมาะสม

4.1.4 การสร้างโปรแกรมระบบ (Construction , Coding , Implementation)

โดยส่วนใหญ่กิจกรรมในเฟสนี้จะเป็นการสร้างโปรแกรมหรือการอิมพลีเมนต์ระบบอันเป็นขั้นตอนของ OOP(OO Programming) ซึ่งต้องอาศัยความรู้ในตัวยุทธศาสตร์โปรแกรมที่ใช้ในการสร้างโค้ดในขั้นตอนนี้จะถูกดำเนินการภายหลังจากที่ได้รับข้อมูลการออกแบบเพียงพอ

4.1.5 การทดสอบระบบ (Testing)

เป็นการทดสอบความถูกต้องของระบบที่พัฒนาเพื่อค้นหาข้อผิดพลาดเชิงเทคนิค และการตรวจสอบความสอดคล้องกับความต้องการของผู้ใช้งาน ทั้งนี้การค้นพบข้อผิดพลาดถือว่าเป็นความสำเร็จของการทำงานเฟสนี้ นอกจากนี้ยังเป็นการประเมินความสมบูรณ์ของระบบว่าจำเป็นต้องทำการวิเคราะห์ออกแบบเพิ่มเติมอีกหรือไม่ โดยจะมีการจัดเตรียมข้อมูลที่ใช้สำหรับการทดสอบและประเมินผลลัพธ์เรียกว่า Test Cases ซึ่งใช้ในการทดสอบส่วนต่างๆของระบบในทุกแง่มุมของการทำงานทั้งหมดที่เป็นไปได้ ผลของการทดสอบจะถูกบันทึกลงรายงานการทดสอบซึ่งรวมถึงการบรรยายรายละเอียดข้อผิดพลาดที่ปรากฏเพื่อทำการแก้ไขต่อไป

4.2 Unified Modeling Language

Unified Modeling Language หรือ UML เป็นภาษาในการโมเดลมาตรฐาน วัตถุประสงค์เบื้องต้นเป็นการพัฒนากระบวนการพัฒนาซอฟต์แวร์เชิงวัตถุที่เป็นหนึ่งเดียวกัน (Unified Method) ต้องการสร้างโมเดลในการพัฒนาที่เข้าใจได้และสร้างได้ง่าย และสามารถนำไปใช้ได้กับทุกระบบ ใน UML มีโมเดลที่สื่อสารด้วยภาพได้สำหรับระบบหลายๆโมเดล โดยแต่ละโมเดลก็จะแสดงมุมมองต่อระบบที่ไม่เหมือนกัน ประกอบไปด้วยไดอะแกรมต่างๆ 8 ไดอะแกรมให้เลือกใช้ได้ตามความเหมาะสม

- ยูสเคสไดอะแกรม (Use Case Diagram) ใช้ในการ โมเดลฟังก์ชันการทำงานของระบบ
- คลาสไดอะแกรม (Class Diagram) ใช้ในการ โมเดลคลาสต่างๆที่จำเป็นในระบบ
- แอ็กทิวิตีไดอะแกรม (Activity Diagram) มีหลักการเดียวกันกับโฟลว์ชาร์ต (Flowchart)
- สเตตชาร์ตไดอะแกรม (Statechart Diagram) ใช้สำหรับแสดงถึงสถานะของออบเจกต์ในระหว่างการทำงาน
- คอลเลบอเรชันไดอะแกรม (Collaboration Diagram) ใช้ในการแสดงการทำงานร่วมกันของออบเจกต์ในระบบ
- ซีเควนซ์ไดอะแกรม (Sequence Diagram) ใช้ในการ โมเดลกิจกรรมต่างๆที่เกิดขึ้นกับออบเจกต์ในระบบ
- คอมโพเนนต์ไดอะแกรม (Component Diagram) ใช้สำหรับสร้าง โมเดลของคอมโพเนนต์ในระบบ
- ดีพลอยเมนต์ไดอะแกรม (Deployment Diagram) ใช้แสดงการติดตั้งใช้งานส่วนประกอบต่างๆของระบบ

4.3 ไดอะแกรมที่เป็นโครงสร้างแบบสถิต (Static Diagram)

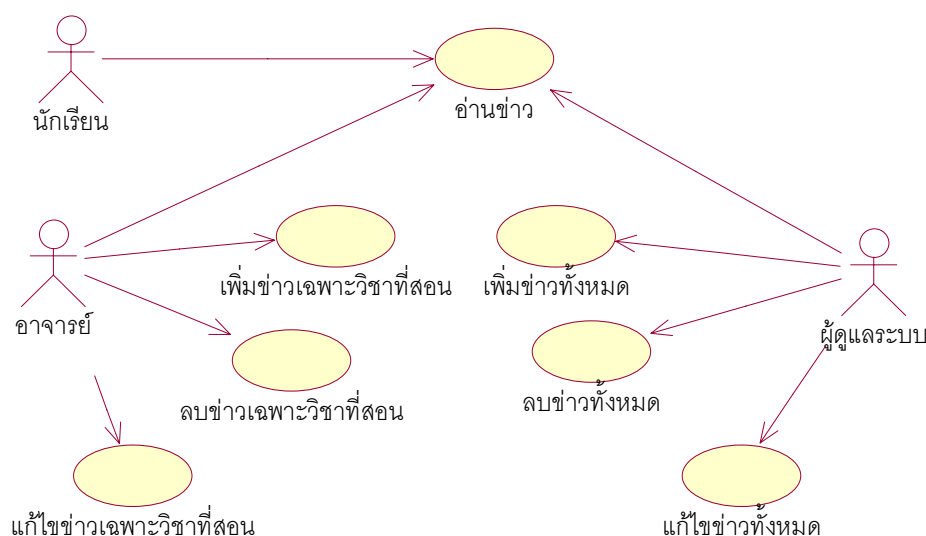
คือเป็นโครงสร้างการทำงานที่หยุดนิ่ง ไม่มีการเปลี่ยนแปลงตามเวลา ซึ่งได้แก่ ยูสเคสไดอะแกรม (Use Case Diagram) และคลาสไดอะแกรม (Class Diagram)

4.3.1 ยูสเคสไดอะแกรม (Use Case Diagram)

ในการสร้างยูสเคสไดอะแกรมสิ่งสำคัญ คือ การค้นหาว่าระบบทำอะไรได้บ้าง โดยไม่สนใจว่าข้างในสิ่งที่ระบบต้องทำได้เหล่านั้นมีกลไกการทำงานอย่างไร หรือใช้เทคนิคการสร้างอย่างไร เปรียบเสมือนเป็น” กล่องดำ ”(Black Box)

ส่วนประกอบของยูสเคสไดอะแกรมจะมีอยู่ 3 ส่วนคือ

- ยูสเคส (Use Case) คือความสามารถหรือฟังก์ชันที่ระบบซอฟต์แวร์จะต้องทำได้ เช่น ค้นหาข้อมูล หนังสือ ยืมหนังสือ คืนหนังสือ บันทึกรายการหนังสือ ลบรายการหนังสือ ฯลฯ
- แอ็กเตอร์ (Actor) คือ ผู้ที่กระทำกับยูสเคสหรือใช้งานยูสเคสนั้นๆ เช่น นักศึกษา(สามารถยืม คืน หนังสือ) บรรณารักษ์(สามารถเพิ่ม ลบรายการหนังสือ) เป็นต้น
- เส้นแสดงความสัมพันธ์ (Relationship) เป็นการเชื่อมโยงระหว่างยูสเคสกับแอ็กเตอร์



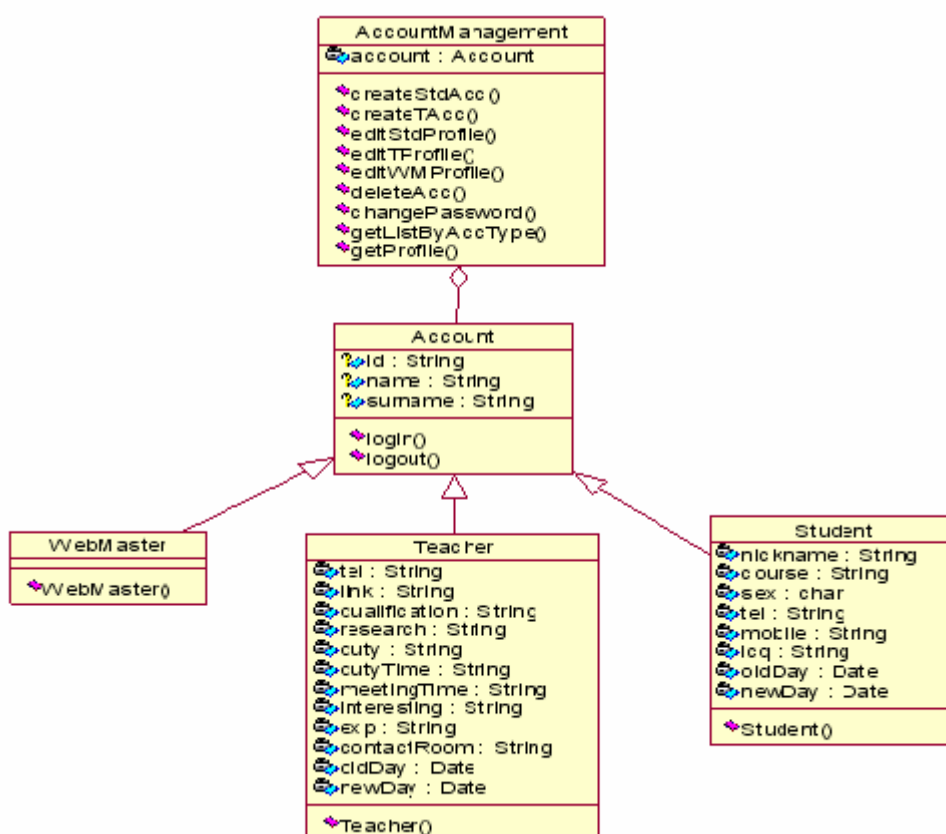
รูปที่ 4-1 แสดง ยูสเคสไดอะแกรม

จากรูปจะพบว่า มีนักเรียน อาจารย์ และผู้ดูแลระบบ เป็นแอ็กเตอร์ โดยแอ็กเตอร์เหล่านี้จะใช้งานหรือกระทำกับยูสเคสที่แอ็กเตอร์ชี้เข้าไป จะมีตัวหนังสือที่แอ็กเตอร์และยูสเคสเพื่อบ่งบอกชื่อของแต่ละองค์ประกอบ

4.3.2 คลาสไดอะแกรม (Class Diagram)

ในการพัฒนาซอฟต์แวร์เชิงวัตถุ จะมีการใช้งานคลาส ออบเจกต์ และมีการสร้างความสัมพันธ์ระหว่างคลาสหรือออบเจกต์เหล่านั้น ดังนั้นการโมเดลระบบเชิงวัตถุจึงมีความจำเป็นอย่างยิ่งที่จะต้องสร้างคลาสไดอะแกรมที่แสดงถึงองค์ประกอบดังกล่าวทั้งหมดอย่างชัดเจน เราเรียกไดอะแกรมนี้ว่า คลาสไดอะแกรม

วัตถุประสงค์ของการสร้างคลาสไดอะแกรมก็เพื่อแสดงถึงโครงสร้างของระบบอันประกอบไปด้วยคลาสต่างๆ และความสัมพันธ์ระหว่างคลาสเหล่านั้น ซึ่งจะถูกใช้เป็นไดอะแกรมหลักในการสร้างไดอะแกรมอื่นๆ อีกหลายประเภท เมื่อนำไปเขียนโค้ดในการแปลงคลาสไดอะแกรมไปเป็นโค้ดนั้นจะค่อนข้างง่ายและตรงไปตรงมาทั้งนี้เนื่องจากภาษาโปรแกรมเชิงวัตถุจะมีวากยสัมพันธ์ (Syntax) ที่ใช้ในการอิมพลีเมนต์คลาสโดยตรง



รูปที่ 4-2 แสดง คลาสไดอะแกรม

จากรูปจะเป็นสัญลักษณ์ของคลาส ตัวอย่างเช่น Account จะมี

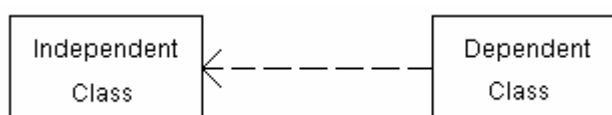
1. ชื่อคลาส คือ Account
2. ส่วนแอตทริบิวต์ จะมี id , name และ surname
3. ส่วนโอเปอเรชัน จะมี login() , logout()

และมีเส้นแสดงความสัมพันธ์กันระหว่างคลาส (Relationships) และรวมถึงความสัมพันธ์ระหว่างออบเจ็กต์ของคลาสด้วย ซึ่งความสัมพันธ์นี้สามารถเป็นไปได้อีก 3 รูปแบบ

4.4 ความสัมพันธ์กันระหว่างคลาส

4.4.1 Dependency

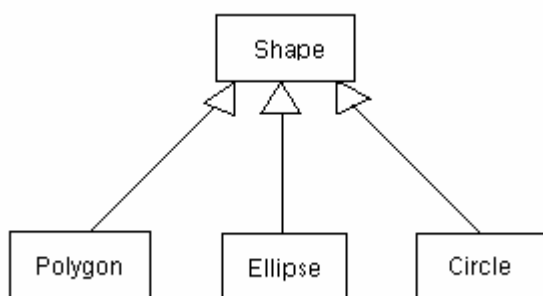
Dependency หรือความสัมพันธ์แบบพึ่งพิง ความสัมพันธ์แบบนี้จะเกิดขึ้นเมื่อการเปลี่ยนแปลงที่เกิดขึ้นกับคลาสที่ถูกพึ่งพิง (Independent Class) จะส่งผลต่อคลาสที่พึ่งพิง (Dependent Class)



รูปที่ 4-3 แสดง ความสัมพันธ์แบบพึ่งพิง

4.4.2 Generalization

Generalization คือความสัมพันธ์ระหว่างซูเปอร์คลาสและซับคลาส



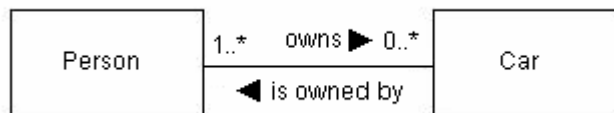
รูปที่ 4-4 แสดง ความสัมพันธ์ระหว่างซูเปอร์คลาสและซับคลาส

4.4.3 Association

Association เป็นความสัมพันธ์อีกชนิดหนึ่งระหว่างคลาสซึ่งแบ่งได้ดังนี้คือ

4.4.3.1 Normal Association

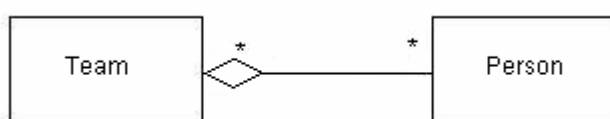
มักใช้ในการโมเดลระบบที่ซับซ้อนโดยเฉพาะระบบสารสนเทศ นอกจากนี้ยังมีการกำหนดปริมาณของคลาสหรือออบเจ็กต์ที่สัมพันธ์กันอยู่ เรียกว่า Multiplicity ซึ่งสามารถกำหนดได้หลายรูปแบบเป็นตัวเลขใส่ไว้ที่ปลายด้านใดด้านหนึ่งของเส้นความสัมพันธ์ เช่นตัวอย่าง สามารถอ่านได้ว่า “A person owns many cars” และ “A car can be owned by many persons”



รูปที่ 4-5 แสดง ความสัมพันธ์ระหว่างคลาสแบบ *Normal Association*

4.4.3.2 Aggregation

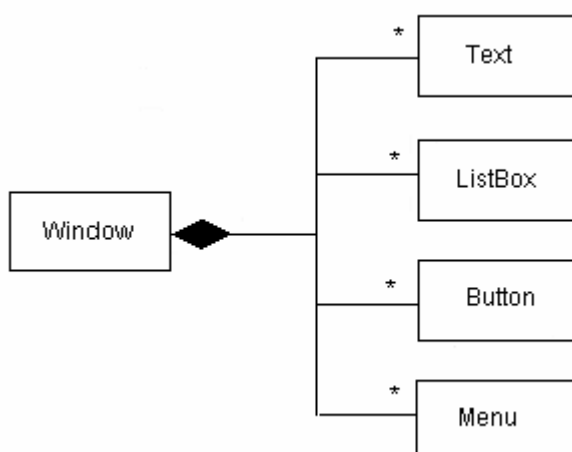
เป็นความสัมพันธ์ระหว่างคลาสหรือออบเจ็กต์ในแง่ของการรวมกันหรือการประกอบกัน สามารถอธิบายได้ตามรูป คือ แต่ละทีมประกอบด้วยสมาชิกหลายคน ในทางกลับกันแต่ละคนอาจสังกัดอยู่มากกว่าหนึ่งทีม



รูปที่ 4-6 แสดง ความสัมพันธ์ระหว่างคลาสแบบ *Aggregation Association*

4.4.3.3 Composition

คล้ายคลึงกับความสัมพันธ์แบบ Aggregation หากแต่คลาสที่เป็นองค์ประกอบจะเป็นส่วนหนึ่งของคลาสใหญ่กว่า และเมื่อคลาสที่ใหญ่กว่าถูกทำลาย คลาสที่เป็นองค์ประกอบก็จะถูกทำลายไปด้วย พร้อมๆกัน เช่นดังรูป คลาสวินโดวส์ จะประกอบไปด้วย คลาสปุ่ม คลาสเมนู เป็นต้น ซึ่งคลาสเหล่านั้นจะอาศัยอยู่ในวินโดวส์ทั้งหมด และเมื่อวินโดวส์ถูกปิดลง ปุ่ม เมนู ก็จะหายไปพร้อมๆกันด้วย นอกจากนี้ในความสัมพันธ์ชนิดนี้ ค่า Multiplicity ของฝั่งที่ใหญ่กว่าจะต้องเป็น 1 เท่านั้น



รูปที่ 4-7 แสดง ความสัมพันธ์ระหว่างคลาสแบบ *Composition Association*

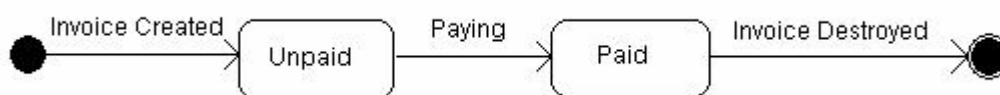
4.5 ไคอะแกรมที่เป็นโครงสร้างแบบไดนามิก (Dynamic Diagram)

จะใช้สำหรับการบรรยายพฤติกรรมของระบบที่มีการเปลี่ยนแปลงตามเวลาในขณะที่ระบบกำลังทำงาน ได้แก่ สเตตชาร์ตไคอะแกรม (Statechart Diagram) , คอลแลบอเรชันไคอะแกรม (Collaboration Diagram) , ซีควเอนซ์ไคอะแกรม (Sequence Diagram) และแอ็กทิวิตีไคอะแกรม (Activity Diagram) ทั้งหมดนี้จะเรียกโดยรวมว่า บีเฮฟเวยอร์ไคอะแกรม (Behavior Diagram) ซึ่งก็คือ ไคอะแกรมที่บ่งบอกพฤติกรรมของระบบ

4.5.1 สเตตชาร์ตไคอะแกรม (Statechart Diagram)

จะบ่งบอกถึงสถานะต่างๆของคลาสในระบบว่ามีสถานะอะไรบ้าง ซึ่งเมื่อเวลาผ่านไป สถานะของคลาสจะสามารถเปลี่ยนไปได้ พฤติกรรมหรือสถานะต่างๆเหล่านี้ย่อมเปลี่ยนไปได้เสมอ เช่นในระบบลิฟต์ ณ เวลาหนึ่งลิฟต์กำลังขึ้นนั่นคือ อยู่ในสถานะขึ้น (State Up) ถ้าลิฟต์กำลังลง ก็คืออยู่ในสถานะลง (State Down) เป็นต้น

สเตตชาร์ตไคอะแกรมในยูเอ็มแอลจะมีจุดเริ่มต้นสถานะและจุดสิ้นสุดสถานะ เส้นลูกศรที่ชี้จากสถานะหนึ่งไปยังอีกสถานะหนึ่ง สามารถเขียนคำอธิบายเหตุการณ์ที่ทำให้เปลี่ยนแปลงสถานะตรงลูกศรได้ จากรูปจะเป็นตัวอย่างแสดง สเตตชาร์ตไคอะแกรมของคลาสใบแจ้งราคาสินค้า (Invoice) ซึ่งมีสถานะคือยังไม่ได้จ่าย (Unpaid) และจ่ายแล้ว (Paid)

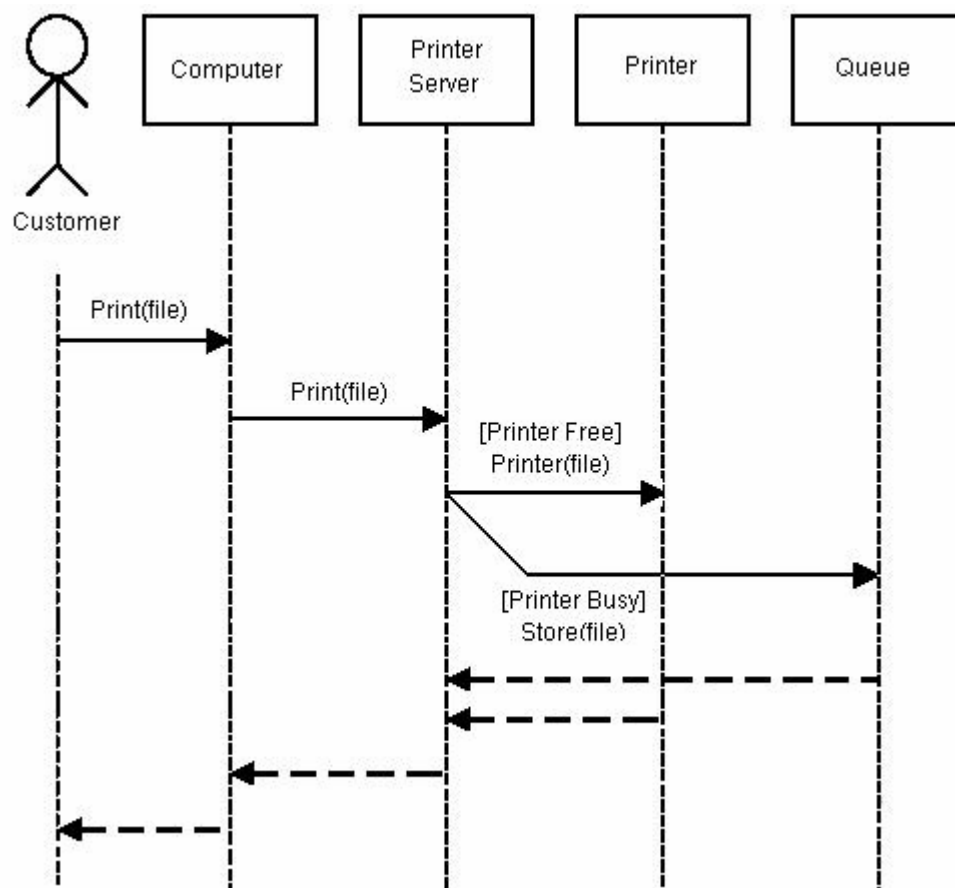


รูปที่ 4-8 แสดง สเตตชาร์ตไคอะแกรม

4.5.2 ซีควเอนซ์ไคอะแกรม (Sequence Diagram)

ซีควเอนซ์ไคอะแกรมจะบอกว่าใน Use Case นั้น วัตถุแต่ละตัวจะติดต่อสื่อสารกันอย่างไร มีขั้นตอนการทำงานอย่างไร โดยจะเน้นไปที่แกนเวลาเป็นสำคัญ ถ้าเวลาเปลี่ยน ขั้นตอนการทำงานจะเปลี่ยน โดยมีแอ็กเตอร์ เป็นผู้กระทำเริ่มต้น

ซีควเอนซ์ไคอะแกรมจะมีแกนสมมติ 2 แกนคือแกนนอน และแกนตั้ง แกนนอนจะแสดงขั้นตอนการทำงานหรือการส่งเมสเสจ (message) ระหว่างวัตถุ โดยแต่ละวัตถุจะส่งข้อมูลถึงกันว่าต้องทำอะไร เมื่อใด ส่วนแกนตั้งเป็นแกนเวลา แกนนอนและแกนตั้งต้องสัมพันธ์กัน

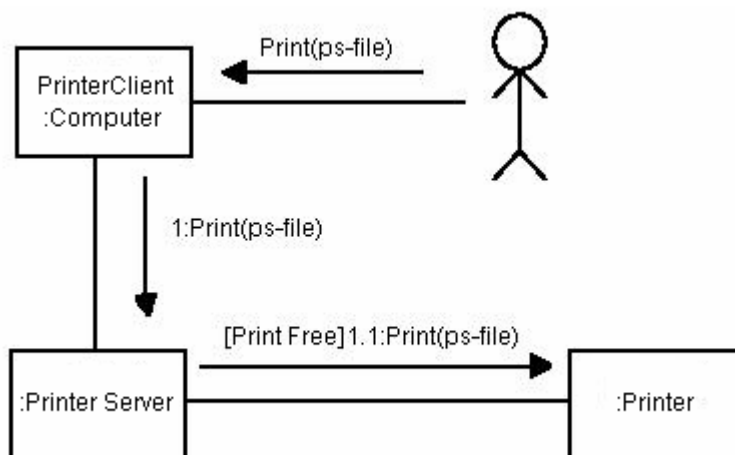


รูปที่ 4-9 แสดงซีเควนซ์ไดอะแกรม

จากรูปเป็นการแสดงขั้นตอนการทำงานของเครื่องพิมพ์เอกสาร โดยเครื่องพิมพ์ เริ่มต้นคอมพิวเตอร์ ส่งพิมพ์ไฟล์ ก็ส่งข้อมูลไปยังเซิร์ฟเวอร์ ต่อจากนั้นเซิร์ฟเวอร์จะดูว่าเครื่องพิมพ์ว่างหรือไม่ ถ้าว่างก็จัดการพิมพ์ ถ้าไม่ว่างก็รอคิวไว้ก่อน เมื่อเครื่องพิมพ์ทำการพิมพ์แล้วก็ส่งกลับไปจนถึงจุดเริ่มต้นคือคอมพิวเตอร์

4.5.3 คอลเลบอเรชันไดอะแกรม (Collaboration Diagram)

คอลเลบอเรชันไดอะแกรมจะมีหน้าที่เดียวกับซีเควนซ์ไดอะแกรม แต่จะไม่แสดงถึงแกนเวลาอย่างชัดเจน ยกเว้นการตอบโต้กันระหว่างออบเจกต์



รูปที่ 4-10 คอลแลบอเรชันไดอะแกรม

จะแสดงถึงกลุ่มของออบเจกต์ที่ทำงานร่วมกันสอดคล้องกับความหมายของชื่อไดอะแกรม ลูกศรจะชี้ไปทางเดียวกัน ไม่มีการชี้ย้อนกลับในเส้นเดียวกัน

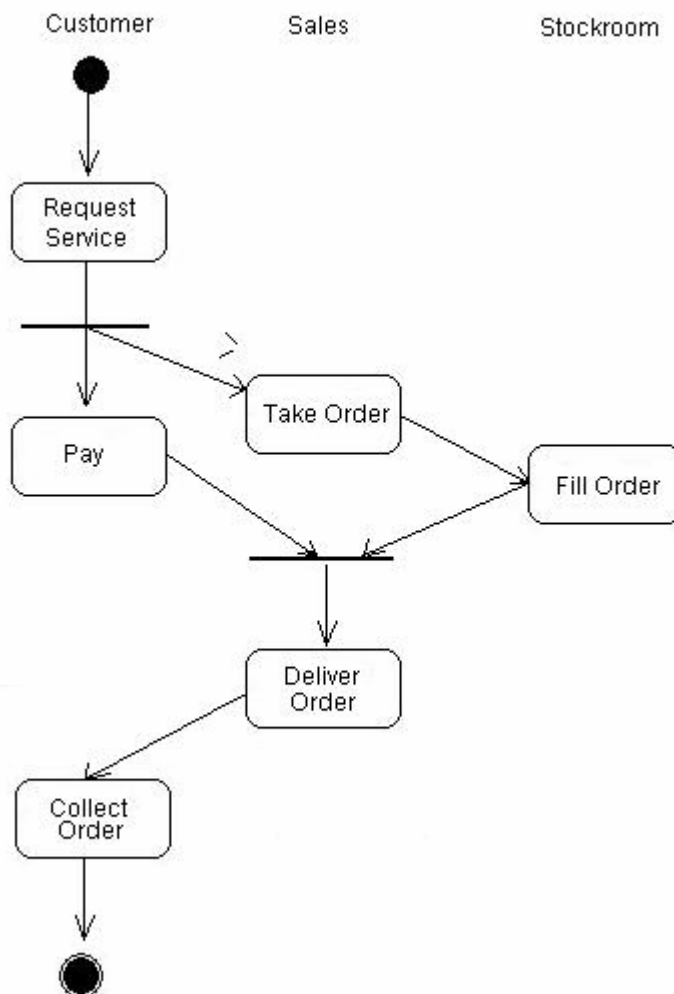
4.5.4 แอ็กทิวิตีไดอะแกรม (Activity Diagram)

แอ็กทิวิตีไดอะแกรมจะแสดงขั้นตอนการทำงานของ Use Case เช่นเดียวกับชีแวนต์ไดอะแกรม และคอลแลบอเรชันไดอะแกรม แต่จะเน้นไปที่งานย่อยของวัตถุ ซึ่งการเจาะจงไปที่งานๆหนึ่งของวัตถุ เหมือนกับสแตตชาร์ตไดอะแกรมที่แสดงสถานะของวัตถุ แต่จริงๆแล้วแอ็กทิวิตีไดอะแกรมต่างกับสแตตชาร์ตไดอะแกรมตรงที่ แอ็กทิวิตีไดอะแกรมจะเปลี่ยนสถานะได้โดยไม่ต้องมีเหตุการณ์ที่กำหนดไว้ในไดอะแกรมมาก่อน แต่มันจะเปลี่ยนสถานะเองตามกระบวนการทำงานคล้ายกับ Flowchart นั้นเอง

ตัวอย่างรูปด้านล่างเป็นการกำหนดว่างานในแต่ละเลนนั้นเกิดขึ้นกับออบเจกต์อะไร หรือกล่าวอีกนัยหนึ่งว่าเป็นการแสดงถึงกิจกรรมที่เกิดขึ้นกับออบเจกต์ที่เป็นเจ้าของเลนนั้น ๆ

ข้อดีของแอ็กทิวิตีไดอะแกรมคือ สามารถแสดงถึงการทำงานในวัตถุนั้นๆอย่างละเอียดคล้าย Flowchart และมีการแบ่งแยกหมวดหมู่ตามออบเจกต์

แอ็กทิวิตีไดอะแกรมยังเหมาะกับการเขียนโมเดลในเชิงธุรกิจเพื่อให้ทราบกระแสการทำงาน (Workflow) ได้ สามารถช่วยแยกแยะผู้รับผิดชอบแต่ละงานได้ว่าใครควรจะเป็นคนทำงานในหมวดหมู่ใด



รูปที่ 4-11 แอ็กทิวิตีไดอะแกรม (Activity Diagram)

4.6 อิมพลีเมนเตชันไดอะแกรม (Implementation Diagram)

จะประกอบไปด้วยสัญลักษณ์ที่ใช้แสดงถึงโครงสร้างของซอร์สโค้ดหรือไฟล์ (ซอฟต์แวร์) และโครงสร้างของส่วนประกอบที่เชื่อมต่อกันในระบบ (ฮาร์ดแวร์)

กลุ่มอิมพลีเมนเตชันไดอะแกรม ประกอบไปด้วย 2 ไดอะแกรมคือ คอมโพเนนต์ไดอะแกรม (Component Diagram) และดีพลอยเมนต์ไดอะแกรม (Deployment Diagram)

4.6.1 คอมโพเนนต์ไดอะแกรม (Component Diagram)

คอมโพเนนต์ไดอะแกรมจะแสดงความสัมพันธ์ที่เชื่อมต่อกันระหว่างซอฟต์แวร์คอมโพเนนต์ในระบบว่าประกอบไปด้วยไฟล์อะไรบ้าง ซึ่งอาจจะเป็น ไฟล์ซอร์สโค้ด (Source Code) ไฟล์ไบนารีโค้ด (Binary Code) และไฟล์เอ็กเซคิวต์ (Executable Code)

การตั้งชื่อของคอมโพเนนต์ในคอมโพเนนต์ไดอะแกรมจะใช้ชื่อของคลาสจากคลาสไดอะแกรม ไม่ใช่ชื่อของอินสแตนซ์ (Instance) แต่ดีพลอยเมนต์ไดอะแกรมจะแสดงด้วยชื่อของอินสแตนซ์

4.6.2 ดีพลอยเมนต์ไดอะแกรม (Deployment Diagram)

ดีพลอยเมนต์ไดอะแกรมแสดงการเชื่อมต่อของอุปกรณ์ฮาร์ดแวร์ในระบบและมักใช้ร่วมกับคอมโพเนนต์ไดอะแกรม โดยข้างในฮาร์ดแวร์อาจประกอบไปด้วยซอฟต์แวร์คอมโพเนนต์ ดีพลอยเมนต์ไดอะแกรมจะแสดงอยู่ในรูปของอินสแตนซ์ และแสดงในช่วงเวลาของการรัน (Run-Time) หรือระหว่างการเธิชชีวิต ดังนั้นคอมโพเนนต์ของระบบที่ไม่ได้ใช้สำหรับรัน (เพราะถูกคอมไพล์ไปแล้ว เช่นไปไฟล์ชอร์ตโค้ด) จะไม่ปรากฏในไดอะแกรมนี้ แต่จะมีในคอมโพเนนต์ของไฟล์ที่ใช้ทำงานจริง ๆ เท่านั้น